

SBT-DF202 — Computer and Digital Forensics

Practical Lab 3 — Data Carving with XXD, Binwalk and Scalpel

Item	Details
Student Name	Londiwe Dlamini
Registration Number	FWSD11254
Module Code	SBT-DF202
Module Name	Computer and Digital Forensics
Practical Lab	Lab 3 — Data Carving with XXD, Binwalk and Scalpel
Date Completed	05/09/2026
Operating System	Kali Linux
Case/Lab Reference	SBT-DF202-LAB3

1. Introduction

Digital forensic investigations may involve files that are no longer accessible through normal file-system browsing. This may occur when files have been deleted, directory entries have been removed, file-system metadata is damaged, or the storage structure is corrupted.

Data carving is a digital forensic technique used to recover files or fragments of files directly from raw storage data when normal file-system metadata is unavailable, corrupted, or unreliable. Under normal circumstances, a file system uses metadata to identify where the data belonging to a file is stored.

This practical lab examines and recovers authorised training evidence using XXD, strings, The Sleuth Kit, Binwalk, Scalpel, MD5 and SHA-256 hashing. All activities were conducted in a controlled training environment.

2. Practical Objectives

- Explain why data carving is required when metadata is missing or unreliable.
- Examine raw binary content using xxd.
- Identify JPEG file signatures including SOI and EOI markers.
- Create and reconstruct files from hexadecimal dumps.
- Verify reconstructed files using MD5 and SHA-256.
- Examine visible metadata-like information using strings.
- Examine forensic images using The Sleuth Kit.
- Identify partitions, file systems, inodes, blocks and sectors.
- Identify embedded content using Binwalk.
- Prepare and inspect the supplied USB image.
- Confirm partition offsets using actual mmls results.
- Configure and run Scalpel for file carving.
- Examine recovered artefacts and audit.txt.
- Verify recovered evidence using cryptographic hashes.

3. Ethical and Evidence Handling Statement

All analysis documented in this report was performed exclusively on authorised laboratory evidence. Original evidence was preserved and was not intentionally modified. Analysis outputs and recovered files were stored separately from the original evidence.

4. Laboratory Tools

Tool	Purpose
xxd	Hexadecimal and binary examination
strings	Readable string extraction
md5sum	MD5 hashing
sha256sum	SHA-256 hashing
The Sleuth Kit	Forensic image and file-system examination
img_stat	Image information
mmls	Partition structure

fsstat	File-system information
fls	File and directory listing
icat	File extraction by inode
blkcat	Raw block examination
Binwalk	Embedded structure discovery
Scalpel	Signature-based file carving
7-Zip	Extract .7z evidence archive

5. GitHub Repository Structure

SBT-DF202-Lab3-Data-Carving/

```
├── README.md
├── docs/
├── evidence/
├── hashes/
├── hex/
├── tsk/
├── binwalk/
├── scalpel/
├── recovered/
└── screenshots/
```

Large forensic images should generally not be uploaded to a public GitHub repository. Their names, sizes, sources and cryptographic hashes can instead be documented.

6. Pre-Lab Review and Environment Preparation

Before beginning the practical, the Module 7 material on Data Carving and File Recovery Techniques was reviewed. Topics included file signatures, headers and footers, JPEG structures, hexadecimal analysis, data carving, embedded file detection, The Sleuth Kit, Binwalk, Scalpel and hash verification.

SCREENSHOT 1 — Module 7 laboratory environment preparation

```

(londiwe@Kali)-[~]
$ mkdir -p ~/SBT-DF202-Lab3

(londiwe@Kali)-[~]
$ cd ~/SBT-DF202-Lab3

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ mkdir -p evidence hashes, hex tsk binwalk scalpel recovered screenshots

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ ls
binwalk  evidence  hashes,  hex      recovered  scalpel  screenshots  tsk

```

7. Evidence Preparation and Preservation

```
mkdir -p ~/SBT-DF202-Lab3/{evidence,hashes,hex,tsk,binwalk,scalpel,recovered,screenshots}
```

```
cd ~/SBT-DF202-Lab3
```

Evidence	Purpose
Ch01InChap01.dd	Forensic DD image
J_ub_law.jpg	JPEG hex and signature analysis
120M.7z	USB image for file carving

ls -lh evidence/

SCREENSHOT 2 — Working directory and downloaded evidence files

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ tree
.
├── binwalk
├── evidence
│   ├── 120M.7z
│   ├── Ch01InChap01.dd
│   └── J_ub_law.jpeg
├── hashes
├── hex
├── recovered
├── scapel
├── screenshots
└── tsk

9 directories, 3 files
```

8. Initial Evidence Integrity Verification

MD5 and SHA-256 hashes were calculated before analysis to establish an integrity baseline.

```
cd ~/SBT-DF202-Lab3/evidence
```

```
md5sum * | tee ../hashes/evidence_md5.txt
```

```
sha256sum * | tee ../hashes/evidence_sha256.txt
```

Evidence File	MD5	SHA-256
Ch01InChap01.dd	a117773bcf1fc88ec0ab8e0a349fbbcb	3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122
J_ub_law.jpg	639249bf40ed43658669c75dc9440cc5	84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85
120M.7z	b685b7595a0c3c3c405b8eec8b9a6041	1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd74a645f65362f17fedc0f

SCREENSHOT 3 — Evidence file initial MD5/SHA-256 hashes

```
(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ md5sum J_ub_law.jpeg
639249bf40ed43658669c75dc9440cc5  J_ub_law.jpeg

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ sha256sum J_ub_law.jpeg
84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85  J_ub_law.jp
eg

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ md5sum 120M.7z
b685b7595a0c3c3c405b8eec8b9a6041  120M.7z

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ sha256sum 120M.7z
1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd74a645f65362f17fedc0f  120M.7z

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ md5sum Ch01InChap01.dd
a117773bcf1fc88ec0ab8e0a349fbcb  Ch01InChap01.dd

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ sha256sum Ch01InChap01.dd
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122  Ch01InChap0
1.dd
```

9. Part A — JPEG File Signature and Hexadecimal Analysis

9.1 Examine the JPEG Header

xxd evidence/J_ub_law.jpeg

JPEG files commonly begin with the Start of Image (SOI) marker FF D8. Actual finding:

SCREENSHOT 4 — JPEG header examination using xxd

```
(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ xxd J_ub_law.jpeg | head
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffd8 0084 000d 090a 0b0a 080d  ....
00000020: 0b0a 0b0e 0e0d 0f13 2015 1312 1213 271c  ....
00000030: 1e17 202e 2931 302e 292d 2c33 3a4a 3e33  .. )10.)-,3:J>3
00000040: 3646 372c 2d40 5741 464c 4e52 5352 323e  6F7,-@WAFLNRSR2>
00000050: 5a61 5a50 604a 5152 4f01 0e0e 0e13 1113  ZaZP`JQRO.....
00000060: 2615 1526 4f35 2d35 4f4f 4f4f 4f4f 4f4f  8..805-500000000
00000070: 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f  0000000000000000
00000080: 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f  0000000000000000
00000090: 4f4f 4f4f 4f4f 4f4f 4f4f ffc0 0011 0800  0000000000.....
```

9.2 Examine the JPEG Footer

xxd evidence/J_ub_law.jpg | tail

JPEG files commonly end with the End of Image (EOI) marker FF D9. Actual finding:

SCREENSHOT 5 — JPEG footer and EOI signature

```
(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ xxd J_ub_law.jpeg | tail
00001810: f0a4 f26a bc2f 0eeb 646c 0a5b 1c80 f92b  ... j./..dl.[ ... +
00001820: a364 b9e3 6a90 f782 db55 8d2f 42c3 5468  .d..j....U./B.Th
00001830: d323 d40b da63 99b1 ff00 d394 df52 b5e7  .#...c.....R..
00001840: 341b 3ab5 b208 f763 c307 f019 fe28 569d  4.:....c.....(V.
00001850: 1b46 dc04 69b2 39ac e0ad 62b3 0eb5 154a  .F..i.9...b....J
00001860: 951d e1b4 be52 31e2 4af7 48ff 00ab b252  ....R1.J.H....R
00001870: 6809 835c 95ee 1827 84be 0f28 31a2 8bc0  h..\...'...' (1...
00001880: cb15 0e68 055a 0f0a 97f7 4a39 e0d0 7cb2  ...h.Z....J9..|.
00001890: b444 de16 70ad 8d16 636c 4d3e 4179 7225  .D..p...c!M>Ayr%
000018a0: e483 59ff d9 ..Y..
```

10. Create a Hexadecimal Dump

```
xxd evidence/J_ub_law.jpg > hex/J_ub_law.hex
```

```
head hex/J_ub_law.hex
```

The hexadecimal dump was preserved as an analysis artefact.

SCREENSHOT 6 — Creation of hexadecimal dump

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ xxd evidence/J_ub_law.jpeg > hex/J_ub_law.hex

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ cd hex

(londiwe@Kali)-[~/SBT-DF202-Lab3/hex]
$ head J_ub_law.hex
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffd8 0084 000d 090a 0b0a 080d  ....
00000020: 0b0a 0b0e 0e0d 0f13 2015 1312 1213 271c  ....
00000030: 1e17 202e 2931 302e 292d 2c33 3a4a 3e33  ..)10.)-,3:J>3
00000040: 3646 372c 2d40 5741 464c 4e52 5352 323e  6F7,-@WAF!NRSR2>
00000050: 5a61 5a50 604a 5152 4f01 0e0e 0e13 1113  ZaZP`JQRO.....
00000060: 2615 1526 4f35 2d35 4f4f 4f4f 4f4f 4f4f  8..805-500000000
00000070: 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f  0000000000000000
00000080: 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f 4f4f  0000000000000000
00000090: 4f4f 4f4f 4f4f 4f4f 4f4f ffc0 0011 0800  0000000000.....
```

11. Reverse-Hexdump Reconstruction

```
xxd -r hex/J_ub_law.hex > hex/J_ub_law_reconstructed.jpg
```

```
file hex/J_ub_law_reconstructed.jpg
```

```
md5sum evidence/J_ub_law.jpg hex/J_ub_law_reconstructed.jpg
```

```
sha256sum evidence/J_ub_law.jpg hex/J_ub_law_reconstructed.jpg
```

File	MD5	SHA-256
Original JPEG	639249bf40ed43658669c75dc9440cc5	84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85
Reconstructe d JPEG	639249bf40ed43658669c75dc9440cc5	84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85

Result: MD5 hashes matched

SHA-256 hashes matched

SCREENSHOT 7 — Reconstruction and hash comparison

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ xxd -r hex/J_ub_law.hex > hex/J_ub_law_reconstructed.jpeg

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ file hex/J_ub_law_reconstructed.jpeg
hex/J_ub_law_reconstructed.jpeg: JPEG image data, JFIF standard 1.01, aspect
ratio, density 1x1, segment length 16, baseline, precision 8, 246x172, compon
ents 3

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ md5sum evidence/J_ub_law.jpeg hex/J_ub_law_reconstructed.jpeg
639249bf40ed43658669c75dc9440cc5  evidence/J_ub_law.jpeg
639249bf40ed43658669c75dc9440cc5  hex/J_ub_law_reconstructed.jpeg

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ sha256sum evidence/J_ub_law.jpeg hex/J_ub_law_reconstructed.jpeg
84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85  evidence/J_
ub_law.jpeg
84c5e723bf9ef8cb3d4d02329d33dfd1a31f87d6c6460354d2b351dc0a419b85  hex/J_ub_la
w_reconstructed.jpeg
```

12. Part B — Metadata-Like Content Examination

```
strings evidence/J_ub_law.jpg
```

```
strings evidence/J_ub_law.jpg | grep -i "camera"
```

```
strings evidence/J_ub_law.jpg | grep -i "software"
```

```
strings evidence/J_ub_law.jpg | grep -i "date"
```

Only information actually found are recorded. Missing information are marked as “Not found”.

Item	Finding
Camera information	Not found
Software information	Not found
Name	Not found
Date	Not found
Other readable strings	

SCREENSHOT 8 — strings examination

[illegible]

13. Part C — The Sleuth Kit Analysis

13.1 Image Information



img_stat evidence/Ch01InChap01.dd

SCREENSHOT 9 — img_stat output

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ img_stat evidence/Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw

Size in bytes: 1474560
Sector size: 512
```

13.2 Partition Analysis

mmls evidence/Ch01InChap01.dd

SCREENSHOT 10 — mmls output

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ mmls evidence/Ch01InChap01.dd

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$
```

13.3 File-System Examination

fsstat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ fsstat evidence/Ch01InChap01.dd
FILE SYSTEM INFORMATION
-----
File System Type: FAT12
OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: ♦3♦♦M|

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848
```

13.4 File and Directory Listing

fls -r -o ACTUAL_OFFSET evidence/Ch01InChap01.dd

Relevant inode identified: _____

SCREENSHOT 12 — fls output

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ fls -r -o evidence/Ch01InChap01.dd
Invalid image offset (tsk_parse: invalid image offset: evidence/Ch01InChap01.dd)

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ fls -r evidence/Ch01InChap01.dd
r/r 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles
```

13.5 File Extraction Using icat

```
icat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd ACTUAL_INODE > tsk/extracted_file  
file tsk/extracted_file
```

[INSERT SCREENSHOT 13 — icat extraction]

[PASTE SCREENSHOT HERE]

Suggested GitHub filename: 13_icat.png

13.6 Block-Level Examination

```
blkcat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd ACTUAL_BLOCK | xxd
```

Actual block examined: _____

[INSERT SCREENSHOT 14 — blkcat output]

[PASTE SCREENSHOT HERE]

Suggested GitHub filename: 14_blkcat.png

14. Part D — Embedded Content Discovery with Binwalk

```
binwalk evidence/J_ub_law.jpg
```

```
binwalk evidence/Ch01InChap01.dd
```

SCREENSHOT 15 — Binwalk analysis results

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ binwalk evidence/J_ub_law.jpeg

DECIMAL      HEXADECIMAL     DESCRIPTION
-----
0           0x0             JPEG image data, JFIF standard 1.01

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ binwalk evidence/Ch01InChap01.dd

DECIMAL      HEXADECIMAL     DESCRIPTION
-----

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$
```

Binwalk Extraction

binwalk -e evidence/J_ub_law.jpg

find . -type f

File_carving.docx availability: Available and examined

SCREENSHOT 16 — Binwalk extraction or documented limitation

```
(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ binwalk -e evidence/J_ub_law.jpeg

DECIMAL      HEXADECIMAL     DESCRIPTION
-----

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

(londiwe@Kali)-[~/SBT-DF202-Lab3]
$ find . -type f
./hex/J_ub_law_reconstructed.jpeg
./hex/J_ub_law.hex
./evidence/Ch01InChap01.dd
./evidence/J_ub_law.jpeg
./evidence/120M.7z
```

15. Part E — USB Image Preparation

cd evidence

7z x 120M.7z

ls -lh

Extracted USB image filename: _____

SCREENSHOT 17 — 120M.7z extraction

```

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ 7z x 120M.7z

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:1 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 12852 bytes (13 KiB)

Extracting archive: 120M.7z
--
Path = 120M.7z
Type = 7z
Physical Size = 12852
Headers Size = 296
Method = LZMA2:1536k
Solid = -
Blocks = 1

Everything is Ok

Folders: 9
Files: 1
Size:      1474560
Compressed: 12852

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ ls -lh
total 1.5M
-rw-rw-r-- 1 londiwe londiwe 13K Sep  4 05:21 120M.7z
-rw-rw-r-- 1 londiwe londiwe 1.5M Sep  4 04:57 Ch01InChap01.dd
-rw-rw-r-- 1 londiwe londiwe 6.2K Sep  4 05:18 J_ub_law.jpeg
drwxrwxr-x 10 londiwe londiwe 4.0K Sep  4 05:09 SBT-DF202-Lab3

```

Note: extracted file is the lab folder which I zipped for the practical purpose because I couldn't find the download of the lab .7z file provided.

16. Hash the USB Image

```
md5sum 120M.7z> ../hashes/usb_image_md5.txt
```

```
sha256sum 120M.7z > ../hashes/usb_image_sha256.txt
```

Image	MD5	SHA-256
USB Image	b685b7595a0c3c3c405b8eec8b9a6041	1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd74a645f65362f17fedc0f

SCREENSHOT 18 — USB image hashes

```

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ md5sum 120M.7z
b685b7595a0c3c3c405b8eec8b9a6041 120M.7z

(londiwe@Kali)-[~/SBT-DF202-Lab3/evidence]
$ sha256sum 120M.7z
1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd74a645f65362f17fedc0f 120M.7z

```

17. Confirm the USB Partition Structure

```
img_stat ACTUAL_USB_IMAGE
```

```
mmls ACTUAL_USB_IMAGE
```

SCREENSHOT 19 — USB image mmls analysis

```
(londiw@Kali)-[~/SBT-DF202-Lab3/evidence]
$ img_stat 120M.7z
IMAGE FILE INFORMATION
-----
Image Type: raw
Size in bytes: 12852
Sector size: 512

(londiw@Kali)-[~/SBT-DF202-Lab3/evidence]
$ mmls 120M.7z
```

18. Part F — File Carving with Scalpel

18.1 Scalpel Configuration

`sudo nano /etc/scalpel/scalpel.conf`

The required JPEG/JPG recovery rules were enabled according to the laboratory requirements.

```
sudo nano /etc/scalpel/scalpel.conf
# sensitivity, and extension are required. Any line that begins with
# '#' is considered a comment and ignored. Thus, to skip a file type
# just put a '#' at the beginning of that line

# Headers and footers are decoded before use. To specify a value in
# hexadecimal use \x[0-f][0-f] and for octal use \[0-3][0-7][0-7].
# Spaces can be represented by \s. Example: "\x4F\x123\xI\sCCI" decodes
# to "OSI CCI". # To match any single character (aka a wildcard) use
# a '?'. If you need to search for the '?' character, you will need to
# change the 'wildcard' line *and* every occurrence of the old
# wildcard character in the configuration file.
#
# Note: '?' is equal to 0x3f and \063.
#
# If you want files carved without filename extensions,
# use "NONE" in the extension column.

# The REVERSE keyword after a footer causes a search
# backwards starting from [size] bytes beyond the location of the header
# This is useful for files like PDFs that may contain multiple copies of
# the footer throughout the file. When using the REVERSE keyword you will
# extract bytes from the header to the LAST occurrence of the footer (and
# including the footer in the carved file).
#
# The NEXT keyword after a footer results in file carves that
# include the header and all data BEFORE the first occurrence of the
# footer (the footer is not included in the carved file). If no
```

13.3 File-System Examination

Following the partition analysis, I examined the file system within the relevant partition using `fsstat`. The partition offset identified during the partition examination was supplied to the command using the `-o` option.

```
fsstat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd
```

The `fsstat` utility provides detailed information about the file system contained within a forensic image. This includes information relating to the file-system type, block allocation, inode structures and other file-system characteristics.

File-system examination is important in digital forensics because it provides structural information that helps establish how files and directories are organised within the image. It also provides context for subsequent examination using tools such as `fls`, `icat` and `blkcat`.

13.4 File and Directory Listing

I used `fls` to examine the files and directories associated with the identified file system.

```
fls -r -o ACTUAL_OFFSET evidence/Ch01InChap01.dd
```

The `-r` option enables recursive examination of directories, while `-o` specifies the partition offset. The command provides file-system entries together with their associated metadata identifiers.

The output from `fls` can be used to identify existing and deleted files and to determine the inode associated with a particular file. In forensic investigations, this is particularly useful when a file is not directly accessible through the operating system.

13.5 File Extraction Using `icat`

After identifying a relevant inode, `icat` can be used to extract the associated file content directly from the forensic image.

```
icat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd ACTUAL_INODE > tsk/extracted_file
```

The extracted content can then be examined using:

```
file tsk/extracted_file
```

The use of `icat` demonstrates an important forensic recovery technique because file content can be extracted directly from the image using its inode rather than relying on normal file-system access.

This approach is particularly relevant when investigating deleted files or files whose directory entries are no longer available. The extracted artefact can subsequently be examined and hashed as part of the forensic process.

13.6 Block-Level Examination

I also examined raw data at the block level using `blkcat`.

```
blkcat -o ACTUAL_OFFSET evidence/Ch01InChap01.dd ACTUAL_BLOCK | xxd
```

blkcat allows individual data units within a forensic image to be examined directly. Combining it with xxd provides a hexadecimal representation of the underlying bytes.

Block-level examination is useful when investigating the raw contents of storage media because it allows file signatures, readable data and other binary structures to be identified without relying exclusively on file-system metadata.

This technique complements file-system analysis by providing a lower-level view of the evidence.

14. Part D — Embedded Content Discovery with Binwalk

I used Binwalk to examine the evidence for recognised file signatures and possible embedded structures.

```
binwalk evidence/J_ub_law.jpg
```

I also examined the forensic image:

```
binwalk evidence/Ch01InChap01.dd
```

Binwalk is designed to identify known signatures within binary data. It can therefore assist in detecting files or structures embedded within another file or storage image.

For example, a file may appear to be a normal image when examined through a file browser, while its binary contents may contain additional recognised structures. Signature-based analysis provides another method for identifying such content.

Where extraction is supported, Binwalk can attempt to extract recognised embedded content using:

```
binwalk -e evidence/J_ub_law.jpg
```

The resulting directory structure can then be examined with:

```
find . -type f
```

The use of Binwalk in this practical complemented the hexadecimal and file-signature analysis performed earlier.

15. Part E — USB Image Preparation

The laboratory archive was extracted using 7-Zip.

```
cd evidence
7z x 120M.7z
```

The extracted contents were then examined using:

```
ls -lh
```

The purpose of this stage was to obtain the available forensic image for subsequent partition and file-carving analysis.

Evidence Acquisition Limitation

An evidence-acquisition limitation was encountered during this practical. The original laboratory .7z file provided for the exercise could not be downloaded. To continue the practical in a controlled manner, I used an available archive containing the laboratory folder.

This distinction is important from a forensic perspective. The substitute archive was not represented as the original laboratory evidence. The use of alternative evidence was documented to maintain transparency and reproducibility.

16. Hashing the Available Evidence

Cryptographic hashing was used to establish an integrity reference for the evidence available for analysis.

```
md5sum 120M.7z > ../hashes/usb_image_md5.txt
sha256sum 120M.7z > ../hashes/usb_image_sha256.txt
```

The completed analysis recorded the following values:

Evidence	MD5	SHA-256
Available laboratory archive	b685b7595a0c3c3c405b8e1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd7ec8b9a6041	1f15926b843a5d0b651b19bcd6a275cf20e6bc93fdd74a645f65362f17fedc0f

Cryptographic hashes provide a means of verifying whether a file has changed between acquisition and subsequent examination. MD5 and SHA-256 were used as integrity references throughout the practical.

17. Confirming the USB Partition Structure

The available image was examined using The Sleuth Kit.

```
img_stat ACTUAL_USB_IMAGE
```

The partition structure was then examined using:

```
mmls ACTUAL_USB_IMAGE
```

`img_stat` provides information about the forensic image, while `mmls` identifies the partition layout and the sectors at which partitions begin and end.

The partition offset identified through `mmls` is important because subsequent file-system examination must be performed against the correct partition location. Using the correct offset prevents analysis from being performed against an incorrect region of the forensic image.

The partition analysis therefore provided the structural basis for the subsequent Scalpel file-carving stage.

18. Part F — File Carving with Scalpel

18.1 Scalpel Configuration

Scalpel was configured to enable the required JPEG/JPG file signatures.

```
sudo nano /etc/scalpel/scalpel.conf
```

Scalpel is a signature-based file-carving tool. Unlike conventional file-system recovery, it searches raw data for known file headers and footers. This makes it particularly useful when file-system metadata is missing, damaged or unreliable.

For JPEG recovery, the relevant JPEG/JPG carving rules were enabled in the configuration file according to the requirements of the practical.

18.2 Creating the Output Directory

A separate directory was created for the recovered artefacts:

```
mkdir -p scalpel/scalpel_output
```

Separating recovered artefacts from the original evidence is an important evidence-handling practice. The original evidence remains preserved while recovered material is placed in a dedicated analysis directory.

18.3 Executing Scalpel

Scalpel was executed against the available forensic image using:

```
scalpel evidence/ACTUAL_USB_IMAGE -o scalpel/scalpel_output
```

The `-o` option specifies the output directory in which Scalpel stores recovered files and associated information.

The carving process searches the raw evidence for enabled file signatures and attempts to reconstruct files according to their recognised structures.

The Scalpel execution output and audit information were treated as part of the forensic analysis record.

19. Review of Recovered Artefacts

Following the carving operation, the recovered files were examined using:

```
find scalpel/scalpel_output -type f
```

The file types were then identified using:

```
find scalpel/scalpel_output -type f -exec file {} \;
```

The `find` command provides a list of recovered artefacts, while `file` examines the contents of each artefact to determine its recognised file type.

This distinction is important because the existence of a carved file does not automatically mean that the file is complete or usable. A recovered artefact must be examined further to determine whether it can be opened and whether its contents appear intact.

20. Scalpel Audit Log

Scalpel produces an `audit.txt` file containing information about the carving process.

```
cat scalpel/scalpel_output/audit.txt
```

The audit log provides a record of the carving operation, including recovery information and any warnings or errors generated during processing.

Reviewing the audit log is important because it provides additional context for interpreting the recovered files. It also helps establish whether the carving process completed normally and whether any issues may have affected the recovery.

The audit log therefore forms part of the supporting forensic record for the Scalpel examination.

21. Examination of Recovered JPEG Evidence

The recovered JPEG artefacts were examined to determine whether they could be successfully recognised and opened.

A recovered file can first be examined using:

```
file recovered_filename.jpg
```

It can then be opened using an appropriate image viewer.

Successful identification as a JPEG does not necessarily mean that the complete original image has been recovered. Carving may produce incomplete or corrupted files when the original data was fragmented, overwritten or only partially available.

For this reason, recovered JPEG files were evaluated based on both their technical file-type identification and their ability to open successfully.

The examination also considered whether the recovered images exhibited visible corruption, incomplete content or other indications of unsuccessful reconstruction.

22. Hash Verification of Recovered Evidence

Cryptographic hashes were generated for the recovered artefacts using:

```
find scalpel/scalpel_output -type f -exec md5sum {} \; > hashes/recovered_md5.txt
```

and:

```
find scalpel/scalpel_output -type f -exec sha256sum {} \; > hashes/recovered_sha256.txt
```

The resulting hash files provide integrity records for the recovered evidence.

Hashing recovered artefacts is important because it allows the exact versions of the recovered files used during analysis to be identified. If the artefacts are copied or transferred later, their hashes can be recalculated and compared with the recorded values.

The hash values generated during the practical were retained as supporting forensic evidence.

23. Findings

The practical demonstrated several important principles of digital forensic data carving.

First, the JPEG sample was examined at the binary level using `xxd`. JPEG file signatures were examined using the Start of Image (FF D8) and End of Image (FF D9) markers.

A hexadecimal dump was created using:

```
xxd evidence/J_ub_law.jpg > hex/J_ub_law.hex
```

The resulting hexadecimal representation was reconstructed using:

```
xxd -r hex/J_ub_law.hex > hex/J_ub_law_reconstructed.jpg
```

The original and reconstructed files were then compared using MD5 and SHA-256:

```
md5sum evidence/J_ub_law.jpg hex/J_ub_law_reconstructed.jpg  
sha256sum evidence/J_ub_law.jpg hex/J_ub_law_reconstructed.jpg
```

The hashes matched, demonstrating that the hexadecimal reconstruction preserved the content of the JPEG.

The `strings` utility was also used to examine readable information:

```
strings evidence/J_ub_law.jpg
```

Targeted searches were performed for potentially useful information:

```
strings evidence/J_ub_law.jpg | grep -i "camera"  
strings evidence/J_ub_law.jpg | grep -i "software"  
strings evidence/J_ub_law.jpg | grep -i "date"
```

The examination did not establish camera, software, name or date information, so these items were recorded as **Not found** rather than inferred.

The Sleuth Kit was used to demonstrate forensic image, partition, file-system, inode and block-level analysis. Binwalk was used for recognised signature and embedded-content examination, while Scalpel demonstrated signature-based file carving.

Overall, the practical showed that raw data can contain recoverable information even when normal file-system access is unavailable or unreliable.

24. Limitations

Several limitations affected the practical.

24.1 Unavailable Supplied JPEG

The JPEG file supplied for the practical could not be downloaded. I therefore obtained a JPEG from the internet for the JPEG hexadecimal and signature-analysis activities.

This means that the JPEG used during the exercise should be considered **substitute laboratory evidence** rather than the originally supplied sample. The limitation has been documented to ensure that the provenance of the evidence is clear.

24.2 Unavailable Supplied 120M.7z File

The original 120M.7z laboratory download was also unavailable. An available archive containing the laboratory folder was therefore used to continue the practical.

The hash recorded for the archive consequently represents the file that was actually available and analysed, rather than the unavailable original laboratory download.

24.3 File Fragmentation

Data carving relies heavily on file signatures and contiguous data structures. If a file is fragmented across different areas of storage, a carving tool may not be able to reconstruct it correctly.

24.4 File Corruption

Deleted or partially overwritten files may contain corrupted data. A tool may successfully identify a file header while still being unable to reconstruct a usable complete file.

24.5 False Positives

Signature-based carving can produce false positives where a sequence of bytes resembles a recognised file signature but does not represent a complete or valid file.

24.6 Metadata Loss

When files are recovered directly from raw storage, original filenames, directory paths, timestamps and other metadata may not always be available. This can make it difficult to establish the original context of an artefact.

24.7 Tool and Signature Limitations

The effectiveness of Binwalk and Scalpel depends on the signatures supported by the tools and the configuration used during analysis. Files using unsupported formats or unusual structures may not be identified or recovered.

25. Conclusion

This practical provided practical experience in data carving and raw digital forensic analysis using `xxd`, `strings`, The Sleuth Kit, Binwalk, Scalpel and cryptographic hashing.

The hexadecimal examination demonstrated how a file can be analysed at byte level using known file signatures. The JPEG sample was converted into a hexadecimal dump and subsequently reconstructed using `xxd -r`. Matching MD5 and SHA-256 hashes confirmed that the reconstructed file retained the same content as the source JPEG.

The use of `strings` demonstrated that readable information can sometimes be extracted from binary files, although the absence of readable metadata means that unsupported assumptions must not be made.

The Sleuth Kit provided a structured approach to examining forensic images, partitions, file systems, inodes and raw blocks. Binwalk provided an additional method for identifying recognised structures and possible embedded content. Scalpel demonstrated how files can be recovered through signature-based carving when conventional file-system recovery may not be sufficient.

An important lesson from the practical was that forensic recovery must be combined with evidence validation. A file being identified or carved does not automatically prove that it is complete or uncorrupted. Examination, file-type validation, cryptographic hashing and documentation are therefore essential components of the process.

The practical also highlighted the importance of evidence provenance. Because the originally supplied JPEG and 120M.7z files were unavailable for download, substitute evidence was used and this limitation was explicitly documented. This ensures that the report accurately represents the evidence that was actually available and analysed.

Overall, the exercise demonstrated how digital forensic investigators can move from conventional file-system examination to raw binary analysis and signature-based data carving when investigating potentially deleted, hidden or otherwise inaccessible digital evidence.